1. Two Sum Easy

Acceptance: 56.9% | LeetCode

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

```
**Input:** nums = [2,7,11,15], target = 9
**Output:** [0,1]
**Explanation:** Because nums[0] + nums[1] == 9, we
return [0, 1].
```

Example 2:

```
**Input:** nums = [3,2,4], target = 6
**Output:** [1,2]
```

Example 3:

```
**Input:** nums = [3,3], target = 6
**Output:** [0,1]
```

Constraints:

- `2 <= nums.length <= 104`
- `-109 <= nums[i] <= 109`
- `-109 <= target <= 109`
- **Only one valid answer exists.**

Follow-up: Can you come up with an algorithm that is less than `O(n2)` time complexity?

Claude Opus Solution

Two Sum - Complete Solution Guide

1. Intuition

The key insight is recognizing this as a **complement search problem**. For each number `x` in the array, we're looking for its complement `target - x`.

Instead of searching the entire array for each complement (O(n²)), we can use a **hash map** to remember what numbers we've seen and their indices. This transforms the inner search from O(n) to O(1).

The clever trick: As we iterate, we store `target - num` (what we need) as the key, not `num` itself. When we encounter a number that exists as a key in our map, we've found our pair!

2. Approach (Step-by-Step Pseudocode)

```
1. Create an empty hash map (dictionary)
2. For each index i and number num in the array:
   a. Check if num exists as a key in the hash map
      - If YES: We found our pair! Return [hash_map[num], i]
   b. If NO: Store (target - num) -> i in the hash map
      - This records "if I see (target - num) later, it pairs with index i"
3. (Problem guarantees a solution exists, so we'll always return in step 2a)
```

3. Python Solution

```python
class Solution:
    def twoSum(self, nums: list[int], target: int) -> list[int]:
        # Hash map to store: complement_needed -> index_that_needs_it
        # Key: the number we're looking for
        # Value: the index of the number that would pair with it
        seen = {}

        # Single pass through the array
        for i, num in enumerate(nums):
            # Check if current number is a complement someone was looking for
            if num in seen:
                # Found it! seen[num] is the index of the number that needs 'num'
                # i is the current index
                return [seen[num], i]

            # Current number wasn't found as a complement
            # Record what complement WE need (target - num)
```

```
            # If we see (target - num) later, it should pair with index i
            complement = target - num
            seen[complement] = i

        # Problem guarantees exactly one solution, so we'll never reach here
        return []
```

Walkthrough with Example

```
nums = [2, 7, 11, 15], target = 9

i=0, num=2:
  - Is 2 in seen? No (seen is empty)
  - Store complement: seen[9-2] = seen[7] = 0
  - seen = {7: 0}

i=1, num=7:
  - Is 7 in seen? YES! seen[7] = 0
  - Return [0, 1] ✓
```

4. Complexity Analysis

Time Complexity: O(n)

- We iterate through the array exactly once
- Each hash map lookup and insertion is O(1) average case
- Total: O(n) operations

Space Complexity: O(n)

- In the worst case, we store n-1 entries in the hash map before finding the answer
- The hash map stores at most n key-value pairs

5. Alternative Approaches

Approach 1: Brute Force - O(n^2) time, O(1) space

```
def twoSum(self, nums: list[int], target: int) -> list[int]:
    # Check every pair
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]
```

- Simple but slow for large inputs

Approach 2: Sort + Two Pointers - O(n log n) time, O(n) space

```
def twoSum(self, nums: list[int], target: int) -> list[int]:
    # Store original indices before sorting
    indexed_nums = [(num, i) for i, num in enumerate(nums)]
    indexed_nums.sort()  # Sort by value

    left, right = 0, len(nums) - 1
    while left < right:
        current_sum = indexed_nums[left][0] + indexed_nums[right][0]
        if current_sum == target:
            return [indexed_nums[left][1], indexed_nums[right][1]]
        elif current_sum < target:
            left += 1
        else:
            right -= 1
```

- Useful when you need to find multiple pairs or variations of the problem
- Sorting destroys original indices, so we need to track them

Summary Table

Approach	Time	Space	Notes
Brute Force	O(n^2)	O(1)	Simple, no extra space

Approach	Time	Space	Notes
Hash Map	O(n)	O(n)	**Optimal for single pass**
Sort + Two Pointers	O(n log n)	O(n)	Good for variations

GPT-5 Solution

1) Intuition (Key Insight / Pattern)

This is a classic **"find pair with given sum"** problem.

For each number `x`, we need another number `y` such that:

- `x + y = target` ⟺ `y = target − x` (the **complement**)

If we can quickly check whether the complement has appeared before, we can solve it in one pass. A **hash table (dictionary)** lets us do that in ~O(1) average time per lookup.

---

2) Approach (Step-by-step Pseudocode)

One-pass hash map (store seen numbers → their indices):

```
seen = empty map  // value -> index

for i from 0 to n-1:
    x = nums[i]
    complement = target - x

    if complement in seen:
        return [seen[complement], i]  // found the pair

    seen[x] = i  // remember where we saw x

// problem guarantees exactly one solution, so we will return inside the loop
```

Key detail: insert `x` **after** checking, so you never use the same element twice.

---

3) Python Solution (Clean + Heavily Commented)

```python
from typing import List

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        # seen maps: number_value -> index_where_it_appeared
        # This allows O(1) average-time lookups for complements.
        seen = {}

        # Iterate once through the array
        for i, x in enumerate(nums):
            # The number we would need with x to reach target
            complement = target - x

            # If we've seen the complement before, we found the unique solution
            if complement in seen:
                # Return indices: (index of complement) and (current index i)
                return [seen[complement], i]

            # Otherwise, remember this number's index for future elements
            # We store AFTER the check to avoid using the same element twice.
            seen[x] = i

        # Given constraints say there is exactly one solution, so we should never reach here.
        raise ValueError("No two sum solution exists (contradicts problem constraints).")
```

---

4) Complexity Analysis

- **Time Complexity:** `O(n)` average
  We do one pass over `nums`, and each dictionary lookup/insert is **O(1)** on average.
- **Space Complexity:** `O(n)`
  In the worst case, we store nearly every element in the hash map before finding the answer.

---

5) Alternative Approaches (Brief)

1. **Brute Force (nested loops)**
2. Check all pairs `(i, j)` with `i < j`
3. Time: `O(n^2)`, Space: `O(1)`
4. **Sort + Two Pointers (then map back to indices)**
5. Sort values with original indices, use `l/r` pointers to find sum
6. Time: `O(n log n)` due to sorting, Space: `O(n)` (to keep indices)
7. More complex than the hash-map approach, but useful when you want ordered scanning patterns.

3. Longest Substring Without Repeating Characters Medium

Acceptance: 38.3% | LeetCode

Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:

```
**Input:** s = "abcabcbb"
**Output:** 3
**Explanation:** The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.
```

Example 2:

```
**Input:** s = "bbbbb"
**Output:** 1
**Explanation:** The answer is "b", with the length of 1.
```

Example 3:

```
**Input:** s = "pwwkew"
**Output:** 3
**Explanation:** The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.
```

Constraints:

- `0 <= s.length <= 5 * 104`
- `s` consists of English letters, digits, symbols and spaces.

Claude Opus Solution

Longest Substring Without Repeating Characters

Intuition

The key insight here is that this is a classic **sliding window** problem. We need to find a contiguous substring (window) where all characters are unique.

Key Pattern Recognition: - When we encounter a duplicate character, we don't need to restart from scratch - We can "slide" our window by moving the left boundary past the previous occurrence of the duplicate - Using a hash map to store character positions allows O(1) lookups for duplicates

Think of it as maintaining a "valid window" that expands when possible and contracts when we hit a duplicate.

Approach

```
1. Initialize:
   - A hash map to store each character's most recent index
   - A left pointer (start of window) at 0
   - A variable to track maximum length found

2. Iterate through string with right pointer (end of window):
   - If current character exists in map AND its stored index >= left pointer:
     → This means it's within our current window (duplicate found!)
     → Move left pointer to one position after the duplicate
   - Otherwise:
     → Character is unique in current window
     → Update max_length if current window is larger

   - Always update the character's position in the map

3. Return max_length
```

Python Solution

```python
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        # Hash map: character -> its most recent index in the string
        char_index = {}

        # Left boundary of our sliding window
        left = 0

        # Track the maximum length found so far
        max_length = 0
```

```
        # Right boundary expands as we iterate through the string
        for right, char in enumerate(s):
            # Check if this character creates a duplicate within our window
            # Two conditions must BOTH be true for it to be a "real" duplicate:
            # 1. Character exists in our map (we've seen it before)
            # 2. Its previous position is >= left (it's within current window)
            if char in char_index and char_index[char] >= left:
                # Duplicate found! Shrink window from left
                # Move left pointer just past the previous occurrence
                left = char_index[char] + 1
            else:
                # No duplicate - this window is valid
                # Current window size = right - left + 1
                max_length = max(max_length, right - left + 1)

            # Always update/record current character's position
            # This overwrites any previous position (we only care about most recent)
            char_index[char] = right

        return max_length
```

Walkthrough Example

For `s = "abcabcbb"`:

```
right=0, char='a': No dup, window="a", max=1, map={'a':0}
right=1, char='b': No dup, window="ab", max=2, map={'a':0,'b':1}
right=2, char='c': No dup, window="abc", max=3, map={'a':0,'b':1,'c':2}
right=3, char='a': Dup! 'a' at index 0 >= left(0), move left to 1
                   window="bca", map={'a':3,'b':1,'c':2}
right=4, char='b': Dup! 'b' at index 1 >= left(1), move left to 2
                   window="cab", map={'a':3,'b':4,'c':2}
... and so on
```

Complexity Analysis

Time Complexity: O(n) - We iterate through the string exactly once with the right pointer - Each character is processed once (no nested loops) - Hash map operations (insert/lookup) are O(1) average

Space Complexity: O(min(m, n)) - Where m is the size of the character set and n is string length - In worst case, we store all unique characters - For ASCII: O(128), for extended ASCII: O(256) - More generally: O(min(alphabet_size, n))

Alternative Approaches

1. Basic Sliding Window with Set (Solution 2 in reference)

```
def lengthOfLongestSubstring(self, s: str) -> int:
    char_set = set()
    left = max_length = 0

    for right in range(len(s)):
        # Remove characters from left until no duplicate
        while s[right] in char_set:
            char_set.remove(s[left])
            left += 1
        char_set.add(s[right])
        max_length = max(max_length, right - left + 1)

    return max_length
```

- **Time:** O(2n) = O(n) - each character visited at most twice
- **Space:** O(min(m, n))
- Simpler but slightly less efficient due to incremental left movement

2. Brute Force (Not recommended)

- Check all substrings for uniqueness
- **Time:** $O(n^3)$ - $O(n^2)$ substrings × O(n) to check each

- **Space:** O(n)

The hash map approach (Solution 1) is optimal because it allows us to "jump" the left pointer directly to the correct position instead of incrementing one by one.

GPT-5 Solution

1) Intuition (Key Insight / Pattern)

This is a classic **sliding window** problem:

- We want the longest **contiguous** substring with **all unique** characters.
- If a window `[L..R]` has no duplicates, expanding `R` is always good (may increase answer).
- When adding `s[R]` creates a duplicate, the window becomes invalid. The only way to fix it while keeping contiguity is to **move `L` right** until the duplicate is removed.

Key trick to do this efficiently: - Track the **last index where each character appeared** (`last[ch]`). - When we see a repeated character `ch` at index `R`, we can jump `L` directly to `last[ch] + 1` (but never move `L` backwards).

This makes the solution linear time.

---

2) Approach (Step-by-step Pseudocode)

```
function longestUniqueSubstringLength(s):
    last = empty map  // char -> last seen index
    L = 0
    ans = 0

    for R from 0 to len(s)-1:
        ch = s[R]

        if ch in last AND last[ch] >= L:
            // ch repeated inside current window
            L = last[ch] + 1

        last[ch] = R
        ans = max(ans, R - L + 1)

    return ans
```

---

3) Python Solution (Heavily Commented)

```python
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        # last[c] = the most recent index where character c was seen
        last = {}

        # L = left boundary of the current sliding window [L..R]
        L = 0

        # best answer found so far
        best = 0

        # Expand the window by moving R from left to right
        for R, ch in enumerate(s):

            # If we have seen ch before AND its last position is inside the current window,
            # then ch is a duplicate within [L..R].
            # We must move L to one position right of that last occurrence to restore uniqueness.
            #
            # Important: we only update L if last[ch] >= L
            # because if last[ch] < L, that occurrence is already outside the window.
            if ch in last and last[ch] >= L:
                L = last[ch] + 1

            # Update the last seen index of this character to the current position
            last[ch] = R
```

```
            # Now window [L..R] is guaranteed to have all unique characters
            best = max(best, R - L + 1)

        return best
```

4) Complexity Analysis

- **Time Complexity:** `O(n)`
- Each character is processed once as `R` moves forward.
- `L` only moves forward (never backward), and we "jump" it using `last`, so total movement is also `O(n)`.
- Hash map operations are amortized `O(1)`.
- **Space Complexity:** `O(k)`
- `last` stores at most one entry per distinct character.
- In worst case, `k = min(n, charset_size)`. For general ASCII-like input, it's bounded; for arbitrary chars, it can be up to `n`.

5) Alternative Approaches (Brief)

1. **Sliding Window with a Set (two pointers)**
2. Maintain a set of chars in the window.
3. If duplicate appears, move `L` one step at a time removing chars until valid.
4. Still `O(n)` time, but may do more incremental steps than the "jump" method.
5. **Brute Force**
6. Check all substrings and test uniqueness.
7. Typically `O(n^3)` (or `O(n^2)` with optimization), too slow for `n = 5e4`.
8. **Fixed-size array instead of dict**
9. If character set is known small (e.g., ASCII 128/256), use an integer array for last positions for speed and predictable memory.